JavaScript

Notes By Sir Yasir Ali Lashari

TITANS-SUKKUR

Definition • Syntax • Benefits • React Components

Topic: Template Literals (ES6+)	**Level:** Beginner → Intermediate	**Language:** JavaScript / React

Table of Contents

01 — Definition & Introduction

Template Literals (also called Template Strings) are a modern JavaScript feature introduced in **ES6 (ECMAScript 2015)**. They allow you to embed variables, expressions, and even multi-line text directly inside a string — using **backticks** (`) instead of single or double quotes.

Key Syntax Rule
Use backtick characters ` ` to wrap the string, and ${ } to embed any JavaScript expression.

02 — Real-Life Analogy

Think of a **form letter**: instead of rewriting someone's name 10 times, you write "Dear [NAME]" once and fill it in automatically. Template literals work exactly the same way — you define a string template once and JavaScript fills in the variable values at runtime.

Real Example
Imagine sending a welcome SMS: "Hello Ali, your order #1042 from Karachi is confirmed." — instead of building this by joining 6 string pieces, a template literal writes it in one clean line.

03 — Syntax & Key Features

Basic syntax:

```
// Declare variables
const name = "Ali";
const age = 22;

// Template literal
const msg = `My name is ${name} and I am ${age} years old.`;
```

Three core features:

Variable insertion	`"Hello " + name`	`` `Hello ${name}` ``
Expression eval	`"Age next year: " + (age+1)`	`` `Age next year: ${age+1}` ``
Multi-line string	`"Line1\nLine2"`	`` `Line1\nLine2` (just press Enter) ``

Function call	`"Hi " + greet(name)`	`` `Hi ${greet(name)}` ``

04 — Benefits of Template Literals

Cleaner Syntax No more + signs and quotes. Write strings naturally and readably.	**Inline Expressions** Compute values directly: ${price * 1.1} works inside the string.
Multi-line Support Press Enter inside backticks — no \n escape needed.	**Function Calls** Call any function inside ${}: ${greet(name)} just works.
Improved Debugging Easier to spot errors since the string reads naturally.	**React-Ready** Essential for dynamic classNames, inline styles, and JSX text.

05 — Complete JavaScript Code

```
// User data
const name = "Ali";
const age = 22;
const city = "Karachi";

// OLD WAY (string concatenation — not recommended)
const oldMessage = "My name is " + name + " and I am " + age + " years
old."; console.log(oldMessage);

// TEMPLATE LITERALS (modern way)
const newMessage = `My name is ${name} and I am ${age} years
old.`; console.log(newMessage);

// Expression inside template literal
const calculation = `Next year age will be ${age + 1}`;
console.log(calculation);

// Multi-line string
const address = `Name: ${name}
City: ${city}
```



```
Age: ${age}`;
console.log(address);

// Function using template literal
function greet(userName) {
return `Hello ${userName}, welcome!`;
}
console.log(greet("Ahmed"));
```

06 — Why Use Template Literals in React?

In React, template literals are used extensively because components receive data as **props** — and those props need to be combined with static text to produce meaningful output. Without template literals, JSX strings become unreadable chains of concatenation. Template literals also power:

Dynamic class names `btn ${isActive ? "active" : "inactive"}` **Inline styles** `color: ${theme.primary}; font-size: ${size}px` **URL building** `https://api.example.com/users/${userId}` **Conditional messages** `You have ${count} new message${count !== 1 ? "s" : ""}`

07 — React Folder Structure

```
src/
■■■ components/
■ ■■■ Greeting.jsx
■ ■■■ Product.jsx
■ ■■■ Button.jsx
■ ■■■ UserList.jsx
■■■ App.jsx
```

08 — Greeting.jsx

This component accepts **name** and **city** props and uses template literals to compose a welcome message.


```jsx
// components/Greeting.jsx

function Greeting({ name, city }) {
return (
<div>
<h1>{`Welcome, ${name}!`}</h1>
<p>{`You are visiting from ${city}.`}</p>
</div>
);
}
```

```
export default Greeting;
```

09 — Product.jsx

This component demonstrates **expression evaluation** inside template literals — computing the final price after discount directly in the string.

```
// components/Product.jsx

function Product({ name, price, discount }) {

const finalPrice = price - discount;

return (

<div>

<h2>{`Product: ${name}`}</h2>

<p>{`Original Price: Rs. ${price}`}</p>

<p>{`Discount: Rs. ${discount}`}</p>

<p>{`Final Price: Rs. ${finalPrice}`}</p>

</div>

);

}

export default Product;
```

10 — UserList.jsx (map + template literals)

This component shows how template literals combine perfectly with **Array.map()** to render a dynamic list. Each user's data is embedded directly into the JSX text.

```jsx
// components/UserList.jsx

const users = [
{ id: 1, name: "Ali", city: "Karachi" },
{ id: 2, name: "Ahmed", city: "Lahore" },
{ id: 3, name: "Sara", city: "Islamabad" },
];

function UserList() {
return (
<ul>
{users.map(user => (
<li key={user.id}>
{`${user.name} — ${user.city}`}
</li>
))}
</ul>
);
}

export default UserList;
```

11 — App.jsx (Import & Use All Components)

The root **App.jsx** imports all components and passes props to them. This shows how a real React application assembles multiple template-literal-powered components.

```jsx
// App.jsx

import Greeting from "./components/Greeting";
import Product from "./components/Product";
import UserList from "./components/UserList";

function App() {
return (
<div>



<Greeting name="Ali" city="Karachi" />
<Product name="Laptop" price={85000} discount={5000} />
```

```
<UserList />
</div>
);
}

export default App;
```

12 — Quick Reference Summary

`text`	Basic template literal	`Hello World`
${var}	Embed a variable	`Hi ${name}`
${expr}	Embed an expression	`${age + 1} years`
Multi-line	Use Enter key inside backticks	`Line1\nLine2`
${fn()}	Call a function inside the string	`${greet(name)}`
Conditional	Use ternary operator inside ${}	`${x > 0 ? "pos" : "neg"}`
Nested template	Template inside another template	`outer ${`inner`}`